E ECORIDE Technologies

SPEC-1-Architecture-Guardrails-Development-Standards

Development Standards and Guidelines

Architecture & Engineering Reference

VERSION	STATUS
1.1	Draft
OWNER	LAST UPDATED
Architecture / Product Engineering	2026-05-08

Document Control

Field	Detail
Document Title	Development Standards and Guidelines
Version	1.1
Status	Draft
Owner	Architecture / Product Engineering
Last Updated	2026-05-08

Revision History

Version	Date	Author	Change Description
1.0	2026-05-08	Architecture / Product Engineering	Initial version
1.1	2026-05-08	Architecture / Product Engineering	Added reusable components and platform library guardrails

Sign-Off

Role	Name	Signature	Date
Product Architect	Andrew Borresen		
Lead Developer			
Product Owner			

Table of Contents

1. Purpose 4
2. Core Delivery Principles 5
3. Workspace & Environment Guardrails 6
4. Base44 Guardrails 6
5. Replit Guardrails 7

6. Application Lifecycle Management 8
7. Naming Conventions 9
8. Frontend Guardrails 9
9. Backend Guardrails 10
10. API Standards 11
11. Database Guardrails 11
12. Mobile Approach Guardrails 12
13. AI Application Policy 13
14. Customer-Facing LLM Data Privacy Statement 15
15. Developer-Facing AI Implementation Guardrails 16
16. Security Guardrails 18
17. Observability & Support Standards 19
18. Definition of Ready 19
19. Definition of Done 20
20. Production Release Checklist 20
21. Recommended Technology Defaults 20
22. Simple Rule of Thumb 21
23. One-Page Developer Reminder 21
24. Reusable Components & Platform Library Guardrails 21
Addendum A: Neon Database Guardrails 25

1 Purpose

This document defines the minimum architecture, naming, environment, lifecycle, security, AI and delivery guardrails for teams building applications using low-code, AI-assisted and cloud development platforms.

The intent is to ensure that all applications are:

Easy to understand & maintain
Code and structures are clear, well-named and comprehensible outside any single tool or developer.

Safe to move to production
Environment separation, review gates and release controls are in place before go-live.

Secure by default
Secrets, credentials and sensitive data are protected at every layer from day one.

Ready for future scaling
Architecture boundaries allow growth from MVP monolith through to distributed services.

Not locked to one tool
Code and data are portable outside Base44, Replit or any prototype environment.

AI-governed appropriately
LLM capabilities follow data minimisation, human accountability and auditability rules.

2 Core Delivery Principles

Principle	Standard
API-first	Business logic must sit behind APIs or reusable backend functions, not directly inside UI screens.
Environment separation	Development, staging and production must be separate at all times.
No production experiments	AI-generated or experimental changes must never be applied directly to production.
Secure by default	Secrets, API keys, credentials and tokens must never be hardcoded.
Portable architecture	Code and data structures must be understandable outside Base44 or Replit.
Minimal coupling	Frontend, backend and database concerns must remain logically separated.
Auditability	Important data changes, admin actions and AI-assisted decisions must be traceable.
Human accountability	AI may assist, but humans remain accountable for system behaviour and decisions.

3 Workspace & Environment Guardrails

3.1 Required Environment Model

Every application must use at least the following environments:

DEV
Active development and experimentation

STAGING
Pre-production testing and business validation

PRODUCTION
Live users and live data only

For larger or AI-heavy projects, also add a **SANDBOX** environment — AI prompting, proof-of-concept work and throwaway experiments. Sandbox must never contain production credentials or customer data.

3.2 Environment Rules

Environment	Guardrail
DEV	May contain incomplete features, test data and experimental AI prompts.
STAGING	Must mirror production configuration as closely as practical.

Environment	Guardrail
PRODUCTION	Must only contain reviewed, approved and release-ready functionality.
SANDBOX	Must not contain production credentials, production data or customer confidential information.

4 Base44 Guardrails

Base44 may be used for rapid prototyping, internal workflow apps, MVP UI generation and managed backend experimentation.

4.1 Workspace Naming

```
Pattern:  {client-or-product}-{app-name}-{environment}

Examples:
  my-product-dev
  my-product-staging
  my-product-prod
  internal-crm-dev
```

4.2 Base44 Workspace Rules

Area	Guardrail
DEV workspace	Used for prompting, design changes, schema exploration and feature development.
STAGING workspace	Used for release testing, UAT, workflow validation and demo sign-off.
PROD workspace	Used only for production-ready applications. No exploratory prompting.
Access	Only approved maintainers may edit production apps.
Secrets	Store credentials in workspace/app secrets, never in generated code or prompts.
Data	Do not use production personal data in DEV unless explicitly anonymised.
GitHub	Where available, connect Base44 apps to GitHub before production use so generated code is version-controlled.

4.3 Base44 AI Prompting Rules

- Prompts must be specific and scoped.
- Avoid broad prompts such as: *"redesign the whole app"*.
- Prefer precise prompts such as: *"update the onboarding form only; do not change database schema"*.
- Do not include real customer credentials, personal data, financial records or confidential commercial data in prompts.
- All AI-generated changes must be reviewed before promotion.

5 Replit Guardrails

Replit may be used for fast MVP backend, frontend, API and integration development.

5.1 Replit Workspace Naming

```
Pattern:  {product}-{service}-{environment}

Examples:
  my-product-api-dev
  my-product-admin-web-dev
  my-product-api-staging
  my-product-worker-prod
```

5.2 Replit Environment Rules

Area	Guardrail
DEV Repl	Used for active coding and AI-assisted changes.
STAGING Repl	Mirrors production configuration as closely as possible.
PROD deployment	Must be deployed from reviewed and stable code only.
Secrets	Use Replit Secrets for API keys, tokens and connection strings.
Deployment secrets	Production secrets must be configured separately for deployments; do not assume editor secrets automatically apply to published apps.
Runtime	Web servers must listen on the correct host/port expected by the deployment.
Persistence	Do not rely on local filesystem storage for production data. Use PostgreSQL, object storage or managed services.

5.3 Replit Production Readiness Checklist

- [ ] Code reviewed
- [ ] Secrets moved to environment variables
- [ ] No test credentials in code
- [ ] Database is external and backed up
- [ ] Logging enabled
- [ ] Error handling implemented
- [ ] Authentication and authorisation tested
- [ ] Rate limits applied to public APIs
- [ ] Health check endpoint available
- [ ] Staging deployment tested

6 Application Lifecycle Management

6.1 Standard Lifecycle

Idea / Prompt → Prototype → DEV Build → Code Review → STAGING → UAT / Validation → Production Release → Monitoring & Support

6.2 Release Rules

Rule	Standard
Versioning	Use semantic versioning: major.minor.patch.
Release notes	Every production release must have short release notes.
Rollback	Every production release must have a rollback plan.
Database changes	Schema changes must be reviewed before production.
Breaking changes	APIs must not break existing clients without versioning.
AI changes	AI prompts, model changes or automation rules must be reviewed before production release.

6.3 Branching Model

```
main        → production-ready code
develop     → active integration branch
feature/*   → individual features
fix/*       → bug fixes
release/*   → release preparation

Examples:  feature/user-onboarding   fix/login-validation   release/v1.0.0
```

7 Naming Conventions

Item	Convention	Example
Applications	kebab-case	admin-dashboard
Services	kebab-case	user-service
Repositories	kebab-case	my-platform
Environment variables	UPPER_SNAKE_CASE	DATABASE_URL
Database tables	snake_case plural	user_profiles
API routes	kebab-case / resource-based	/api/v1/user-profiles
React components	PascalCase	UserProfileCard
Functions	camelCase	calculateNetEarnings

Item	Convention	Example
Constants	UPPER_SNAKE_CASE	MAX_RETRY_LIMIT
Files	kebab-case	user-profile-form.tsx
AI prompts	kebab-case	earnings-summary-prompt
Background jobs	kebab-case	report-generation-worker

8 Frontend Guardrails

Recommended stack: React / Vite / TypeScript · Tailwind or component library · API client layer · Role-based routing

8.1 Frontend Standards

Area	Guardrail
Language	TypeScript preferred for maintainability.
Components	Components must be small, reusable and clearly named.
API calls	API calls must be centralised in a service/client layer.
Forms	Validation must exist on both frontend and backend.
State	Keep global state minimal. Use local state where possible.
Errors	Show user-friendly error messages.
Permissions	Hide unauthorised UI actions, but always enforce permissions on backend too.
Styling	Use consistent design tokens for colour, spacing and typography.
AI UI	Clearly indicate when content is AI-generated or AI-assisted where relevant.

8.2 Frontend Folder Pattern

```
src/ components/ pages/ layouts/ hooks/ services/ types/ utils/ constants/ styles/
```

9 Backend Guardrails

Recommended stack: Node.js · Express or NestJS · REST APIs initially · Event-driven patterns where needed

9.1 Backend Standards

Area	Guardrail
API design	Use RESTful resource naming.
Statelessness	Backend must not depend on server-side session state.
Validation	Validate all input at API boundary.
Auth	Authentication and authorisation must be explicit.
Logging	Log important system events, not sensitive data.
Errors	Return consistent error structures.
Secrets	Load secrets from environment variables only.
Business rules	Business rules must be centralised, not duplicated across screens.
Background jobs	Long-running jobs must run as workers/scheduled jobs, not inside request handlers.
AI orchestration	AI calls must be routed through backend services, not directly from frontend/mobile clients.

9.2 Backend Folder Pattern

```
src/ modules/ users/ billing/ reporting/ common/ auth/ validation/ errors/ logging/
config/ database/ jobs/ integrations/ ai/ main.ts
```

10 API Standards

10.1 API Naming — use versioned routes

```
/api/v1/users
/api/v1/accounts
/api/v1/reports
/api/v1/ai/summaries
```

10.2 API Method Rules

Method	Use
GET	Read data
POST	Create data or trigger action
PUT	Replace full resource
PATCH	Update part of resource
DELETE	Delete or deactivate resource

10.3 Standard API Response Shape

```
// Success
{ "success": true, "data": {}, "error": null, "meta": {} }

// Error
{ "success": false, "data": null,
  "error": { "code": "VALIDATION_ERROR", "message": "One or more fields are invalid" },
  "meta": {} }
```

11 Database Guardrails

> **Recommended database:** PostgreSQL — managed Postgres preferred where possible. See Addendum A for Neon-specific guidance.

11.1 Database Naming

Object	Convention	Example
Tables	snake_case plural	user_profiles
Columns	snake_case	created_at
Primary keys	id	id
Foreign keys	{table_singular}_id	user_profile_id
Indexes	idx_{table}_{columns}	idx_bookings_booking_date
Constraints	chk_, fk_, uq_ prefixes	uq_users_email

11.2 Required Columns

Every business table must include: `id` · `created_at` · `updated_at` · `created_by` · `updated_by` · `status`

For multi-tenant systems also include: `tenant_id` · `organisation_id` or `account_id` where relevant.

11.3 Data Rules

- Store all timestamps in UTC.
- Store money in minor units where practical; use ISO currency codes.
- Do not delete important business records unless legally required — prefer soft delete (`deleted_at` or `status`).
- Apply database constraints for critical rules.
- Use migrations for schema changes. Never manually change production schema without a reviewed migration.
- Do not store raw AI prompts/responses containing sensitive information unless explicitly approved.

12 Mobile Approach Guardrails

Phase	Approach
Phase 1	Mobile-first responsive web app or PWA where acceptable
Phase 2	React Native / Expo app for app-store delivery
Phase 3	Native modules only where hardware, biometric or deep device features require them

12.1 Mobile Standards

Area	Guardrail
Framework	React Native with Expo preferred for cross-platform delivery.
Builds	Use separate dev, preview and production build profiles.
Secrets	Never store secrets inside the mobile app bundle.
API URLs	Use environment-specific API URLs.
Offline	Define exactly which features must work offline.
Push notifications	Use a notification abstraction layer.
App stores	Production releases must follow controlled versioning.
Device permissions	Request only permissions that are required.
AI calls	Mobile apps must not call LLM APIs directly using embedded API keys.

13 AI Application Policy

13.1 Purpose

This policy defines how artificial intelligence, machine learning and large language model capabilities may be used in applications developed by the team. It applies to all AI-assisted features, LLM API usage, AI-generated outputs and AI-assisted development tools.

13.2 AI Usage Principles

Principle	Standard
Human accountable	AI may assist decisions, but the application owner remains accountable.
Data minimisation	Send only the data required for the AI task.
No secrets	Never send credentials, tokens, private keys or passwords to an AI model.
No hidden automation	Users must know when AI materially influences outputs or recommendations.
Explainability	AI outputs should be explainable enough for the user or admin to act on.
Reviewability	High-impact AI outputs must be reviewable by a human.

Principle	Standard
Security first	AI features must follow the same security, privacy and audit standards as normal features.

13.3 Approved AI Use Cases

AI may be used without additional approval for: text summarisation · drafting notifications or messages · classification of support issues · data extraction from uploaded documents · matching, reconciliation or anomaly detection · operational recommendations · search and Q&A over approved business data · developer productivity · internal reporting.

> **Additional approval required when AI is used for:** automated financial decisions · user eligibility decisions · suspension or enforcement actions · biometric identification · safety-critical recommendations · legal, medical or regulated advice · any decision that materially affects a person's access, income or rights.

13.4 Prohibited AI Use

- Make final high-impact decisions without human review
- Process secrets or credentials
- Expose one customer's data to another customer (cross-tenant leakage)
- Generate misleading, deceptive or unsupported claims
- Bypass user consent or privacy obligations
- Replace required legal, compliance or safety review
- Store prompts and responses containing sensitive data without approval

13.5 LLM Provider Requirements

LLMs must be accessed through enterprise-grade APIs or approved cloud-hosted AI services.

```
Approved pattern:
  Application backend → AI gateway/service layer → LLM API

Avoid:
  Frontend / mobile app → direct LLM API call
```

14 Customer-Facing LLM Data Privacy Statement

> **Note:** This section contains customer-facing language. Use or adapt this wording in product privacy notices, terms and onboarding flows where AI features are present.

How customer data is used

Customer data is processed only to generate responses, perform the requested function, or support an approved application workflow. Customer data is not used to train, fine-tune or improve public or shared AI models unless explicitly agreed and enabled. Customer data is not visible to other customers.

Data retention

By default, inputs and outputs may be temporarily retained by the AI service provider for security, abuse-prevention, reliability or operational purposes. This retention is time-limited and subject to contractual and

security controls. Zero data retention configurations may be available for customers with heightened compliance requirements.

Data isolation and security

Data is encrypted in transit and at rest where supported by the platform. AI services are accessed through controlled application backends and API gateways. Customer data is not shared with third parties beyond the AI service providers required to deliver the application functionality.

Regulatory alignment

AI usage is designed to align with: data minimisation · purpose limitation · access control · security safeguards · auditability · retention limitation. These principles support POPIA and GDPR obligations where applicable.

15 Developer-Facing AI Implementation Guardrails

15.1 AI Data Classification

Data Type	AI Processing Allowed?	Guardrail
Public content	Yes	Normal controls
Internal business data	Yes	Approved use case required
Customer confidential data	Yes, with controls	Minimise and secure
Personal information	Yes, only if necessary	POPIA/GDPR principles apply
Financial/banking data	Restricted	Redact or avoid where possible
Identity documents	Restricted	Use only for approved workflows
Credentials/secrets	No	Never send
Production database dumps	No	Use anonymised extracts only

15.2 Prompt Rules

- Send only the minimum required fields.
- Prefer IDs, summaries or redacted values over raw sensitive data.
- Never include credentials, tokens, secrets or private keys.
- Store reusable prompts in source control. Version important production prompts.

15.3 Response Rules

- Treat AI output as untrusted until validated.
- Never execute AI-generated code directly.
- Never apply AI-generated database changes automatically.
- Label AI-generated or AI-assisted content where appropriate.

15.4 AI Logging and Audit — log these fields

```
user_id or system actor
feature name
timestamp
model / provider used
request purpose
success / failure status
token / cost metadata where available
```

> **Never log:** raw personal information · identity numbers · banking details · access tokens · private documents · full confidential prompts unless approved. For sensitive workflows, store an audit reference rather than the full prompt.

15.5 AI Environment Separation

```
DEV        → test keys, test data, lower-cost models
STAGING    → production-like model config, non-production data
PRODUCTION → approved model, approved data handling and monitoring

Recommended env vars:  AI_PROVIDER · AI_MODEL_NAME · AI_API_KEY · AI_API_BASE_URL
                       AI_LOGGING_ENABLED · AI_RETENTION_MODE · AI_TIMEOUT_SECONDS · AI_MAX_
```

15.6 AI Cost and Rate Guardrails

Every AI-enabled application must define: maximum tokens per request · maximum requests per user/session/day · timeout limits · retry limits · monthly budget alert · fallback behaviour when the AI provider is unavailable.

> **AI must not be placed inside loops, background jobs or bulk processing** without explicit cost controls.

15.7 AI Security Guardrails

Applications must protect against: prompt injection · data exfiltration through generated responses · insecure tool/function calls · over-permissive retrieval · cross-tenant data leakage · unvalidated structured output.

Minimum controls: input validation · system prompt controls · retrieval scoped by tenant/user · server-side authorisation checks · output validation · rate limiting · audit logging.

15.8 AI Release Checklist

- [ ] AI use case approved
- [ ] Provider approved and model documented
- [ ] Data classification reviewed
- [ ] No secrets sent to AI provider
- [ ] Prompt minimises personal/confidential data
- [ ] Tenant/user permissions enforced before retrieval
- [ ] Output validation implemented

- [ ] Human review included for high-impact decisions
- [ ] AI logs exclude sensitive raw data
- [ ] Cost/rate limits configured
- [ ] Fallback behaviour implemented
- [ ] Customer-facing privacy wording updated if needed
- [ ] Production keys stored securely
- [ ] AI feature tested in staging

16 Security Guardrails

> **Non-negotiable:** The items below are mandatory across all applications and environments without exception.

- No credentials in code. No credentials in AI prompts.
- No production data in screenshots or demos unless approved.
- All APIs must enforce authentication where required.
- All role-based actions must be checked on the backend.
- All sensitive fields must be encrypted or protected appropriately.
- Logs must not contain passwords, tokens, ID numbers, banking details or private documents.
- Use HTTPS only. Apply rate limiting to public APIs.
- Enable audit logging for administrative actions.
- AI-enabled workflows must follow the same security standards as non-AI workflows.

16.2 Secrets Naming

```
DATABASE_URL · JWT_SECRET · OPENAI_API_KEY · AZURE_OPENAI_API_KEY
SENDGRID_API_KEY · STORAGE_ACCESS_KEY
```

17 Observability & Support Standards

Capability	Minimum Standard
Application logs	Enabled for backend and workers
Error tracking	Centralised error capture
API monitoring	Track failures and latency
Audit logs	Admin and critical business actions
AI usage logs	Track model usage, cost and failures without exposing sensitive data
Health checks	/health endpoint
Alerts	Production failures must notify responsible team

Capability	Minimum Standard
Backups	Production database backups must be verified

18 Definition of Ready

A feature is ready for development when all of the following are true:

- Requirement is clear and acceptance criteria exist
- Data model impact is understood
- API and security impact is understood
- AI impact is understood, where relevant
- Environment impact is understood
- Test scenario is defined

19 Definition of Done

A feature is done when all of the following are true:

- Code is reviewed and works in DEV and STAGING
- Validation, permission checks and error handling exist
- Logs are appropriate — no secrets hardcoded
- Database changes are migrated
- AI prompts/outputs reviewed where relevant
- Release notes updated and business owner accepts the feature

20 Production Release Checklist

- [ ] Code merged to production branch
- [ ] Environment variables configured
- [ ] Database migration reviewed
- [ ] Backup confirmed
- [ ] Smoke test completed in staging
- [ ] Rollback plan documented
- [ ] Release notes prepared
- [ ] Monitoring checked
- [ ] AI configuration checked where relevant
- [ ] Responsible owner assigned
- [ ] Business approval received

21 Recommended Technology Defaults

Layer	Preferred Standard
Frontend web	React + TypeScript + Vite
Admin dashboard	React + TypeScript
Backend	Node.js with Express or NestJS
Database	PostgreSQL
Mobile	React Native / Expo
Hosting MVP	Replit / Vercel / managed platform
Production hosting	Cloud-hosted container/serverless environment
Authentication	Managed auth preferred where appropriate
File storage	Managed object storage
AI integration	API-based, server-side orchestration
Observability	Centralised logging and error monitoring

22 Simple Rule of Thumb

Still an MVP?

Build fast, but keep clean boundaries.

Has real users?

Separate environments, protect data, review releases.

Handles money, identity, documents, AI decisions or operational control?

Treat it as production-grade from day one.

23 One-Page Developer Reminder

✓ DO

- Use clear names
- Separate dev, staging and production
- Store secrets securely
- Keep APIs versioned
- Validate backend inputs
- Use migrations for database changes
- Review AI-generated code
- Test before production
- Log important events
- Keep business rules centralised
- Route AI calls through the backend

✗ DO NOT

- Hardcode secrets
- Build directly in production
- Put business rules only in UI
- Use production data in dev
- Skip permissions on backend
- Manually edit production databases
- Let AI rewrite large app sections without review
- Depend on local files for production storage
- Send credentials or personal data to LLMs
- Call LLM APIs directly from mobile or web clients

- Validate AI outputs before saving

24 Reusable Components & Platform Library Guardrails

Development must follow a reuse-first approach. Teams should not reinvent common functions, UI patterns, backend services, integration logic or workflow components when an approved reusable component already exists.

24.1 Reuse-First Principle

Principle	Standard
Never reinvent the wheel	Before building a new function, screen, workflow or service, developers must check the approved component/service catalogue.
Reuse before rebuild	Use an existing reusable component where it satisfies the requirement with reasonable configuration or extension.
Extend responsibly	If an existing component is close but incomplete, extend it in a reusable way rather than creating a duplicate.
Catalogue new assets	If a new reusable component is created, it must be added to the catalogue with ownership, version, purpose, usage notes and examples.
Design for future extraction	Reusable backend services must be modular enough to remain inside a monolith initially and be extracted later.

24.2 Reusable Asset Types

Asset Type	Examples
UI components	Buttons, forms, tables, cards, dashboards, modals, filters, steppers, onboarding flows, document upload patterns
UI page patterns	Admin list/detail pages, approval screens, status pages, workflow dashboards, notification centres
Backend services	Authentication helpers, audit logging, notification sending, file upload, document handling, reporting, scheduling, AI gateway services
Integration adapters	Payment providers, email/SMS providers, LLM providers, storage providers, third-party business systems
Business utilities	Date/time handling, currency handling, status transitions, validation rules, permission checks, error handling
Data patterns	Standard table columns, migration templates, tenant scoping, soft delete patterns, audit trail structures

24.3 Platform Library Structure

```
platform-library/ ui/ components/ layouts/ page-patterns/ design-tokens/ backend/
services/ modules/ integrations/ jobs/ utilities/ database/ migrations/ schema-patterns/
```

```
seed-data/ ai/ prompts/ guardrails/ evaluation-tests/ docs/ catalogue.md usage-examples/
ownership-register.md
```

24.4 Reusable Component Requirements

Requirement	Standard
Clear ownership	Every reusable component must have an owner or responsible team.
Versioning	Reusable components must be versioned so consuming applications can upgrade safely.
Documentation	Each component must include purpose, when to use, when not to use, configuration options and examples.
Tests	Reusable backend services and critical UI components must include tests or validation scenarios.
Configuration-driven	Reusable components must avoid hardcoded customer, product or environment-specific logic.
Security reviewed	Reusable services that handle identity, permissions, documents, money, AI or integrations must be security reviewed.
Backward compatible	Breaking changes must be avoided or released as a new major version.

24.5 Monolith-to-Microservices Readiness

- Keep service boundaries clear and aligned to business capabilities.
- Avoid direct cross-module database writes where a service boundary is expected in future.
- Expose reusable functionality through clear service interfaces or APIs.
- Keep integration adapters separate from business workflows.
- Use events for important lifecycle actions where asynchronous processing may be required later.
- Avoid circular dependencies between modules.

24.6 Reuse Workflow for Developers

1. Check the platform catalogue before building
2. Reuse an existing component if it fits
3. Extend the existing component if the gap is small and generally useful
4. Build a new component only when reuse is not practical
5. Document and catalogue the new component before release
6. Add tests, examples and ownership details
7. Review whether the component should remain local or move into the shared platform library

24.7 Reusable Component Naming

Item	Convention	Example
UI component	PascalCase	UserStatusBadge
UI pattern	PascalCase + Pattern	ApprovalWorkflowPattern
Backend service	kebab-case or PascalCase class	notification-service / NotificationService
Integration adapter	{provider}-{capability}-adapter	sendgrid-email-adapter
Shared utility	camelCase function	calculateDateRange
Prompt template	kebab-case	earnings-summary-prompt
Database migration	timestamp_description	20260508_create_audit_logs

24.8 Reuse Checklist

- [ ] Catalogue checked for an existing component
- [ ] Existing component reuse considered
- [ ] Reason for new build documented if not reused
- [ ] Component has clear ownership and usage examples
- [ ] Component avoids customer-specific hardcoding
- [ ] Component is suitable for use in more than one application
- [ ] Backend component has a clear service boundary
- [ ] Future extraction to service/microservice considered where relevant
- [ ] Component added to platform library or backlog for catalogue inclusion

Addendum A — Neon Database Guardrails

A.1 Purpose

Neon may be used as a PostgreSQL platform for MVP, staging, prototype and production workloads where serverless PostgreSQL, environment isolation and database branching provide delivery speed and operational flexibility.

A.2 When to Use Neon

Use Neon when the application requires: PostgreSQL compatibility · fast MVP setup · separate development/staging/production databases · database branching for feature testing · low-administration database operations · cost-efficient early-stage environments.

Avoid Neon where the system requires strict customer-mandated database hosting, dedicated infrastructure, existing enterprise database contracts, or very high-volume workloads without a clear partitioning and retention strategy.

A.3 Neon Environment Model

```
project: {product-name}
  branch: production      → live users and live data
  branch: staging         → release testing and business validation
  branch: development     → active development and experimentation
  branch: feature/*       → schema experiments and migration testing
```

A.4 Neon Naming Conventions

Object	Pattern	Example
Project names	{product}-{domain}	my-product-platform
Branch names	production / staging / development / feature/{name}	feature/user-onboarding
Database names	{app_name}_{environment}	my_product_prod
Database roles	{app}_{environment}_{access_level}	myapp_prod_readonly

A.5 Neon Guardrails

Area	Guardrail
Production branch	Must be protected. No direct experimental changes.
Development branch	May be used for active development and test data.
Feature branches	Use for schema experiments and migration testing.
Migrations	All schema changes must be migration-based.
Production data	Must not be copied into development unless anonymised.
Access	Developers should not use production admin credentials for normal development.
Secrets	Connection strings must be stored in Replit Secrets, Base44 secrets, CI/CD secrets or secure environment variables.
Backups	Production restore capability must be tested before go-live.
Data residency	Region selection must consider customer, regulatory and latency requirements.
Pooled connections	Use connection pooling where serverless workloads or high concurrency require it.

A.6 Neon Development Workflow

1. Create feature branch from development or staging
2. Apply schema migration and test application changes
3. Review migration with team

4. Apply migration to staging

5. Complete UAT

6. Apply migration to production during release window

7. Monitor logs and database performance

> **Do not** manually change production database schemas through ad hoc SQL unless it is an approved emergency fix.

A.7 Database Standards for Neon/PostgreSQL

Use: snake_case table and column names · plural table names · UUID primary keys where appropriate · UTC timestamps · explicit foreign keys · indexes for high-use filters.

Every business table must include: `id` · `created_at` · `updated_at` · `created_by` · `updated_by` · `status`

Where multi-tenancy applies also include: `tenant_id` · `organisation_id` · `account_id` — use consistently and do not mix naming styles.

A.8 Neon Production Readiness Checklist

- [ ] Production branch exists and is protected
- [ ] Staging and development branches exist
- [ ] Production connection string stored securely — not committed to Git
- [ ] Database migrations are repeatable and restore process tested
- [ ] Access roles separated by environment
- [ ] Read-only role created where reporting access is required
- [ ] Monitoring enabled, slow queries reviewed, indexes added for critical queries
- [ ] Data retention requirements documented